

XIAMEN
UNIVERSITY

COM102 – Programming Language C

Chapter 6: Strings (ii) – Lab

Prepared By: Dr. Najla'a A. M. Draib

In-class Exercises

Count a Specific Character

Write a C program that:

1. Reads a full line of text (including spaces) into a string (max **100** chars).
2. **Reads** one character ch from the user.
3. **Uses** a function `countChar(char s[], char ch)` to count how many times ch appears in the string.
4. **Prints** the count.

Solution:

```c
#include <stdio.h>
int countChar(char s[], char ch) {
    int count = 0;
    int i = 0;

    while (s[i] != '\0') {
        if (s[i] == ch) {
            count++;
        }
        i++;
    }
    return count;
}
int main() {
    char str[100];
    char ch;
    int result;

    // Read a full line of text (including spaces)
    printf("Enter a string: ");
    fgets(str, 100, stdin);

    // Read a character
    printf("Enter a character to count: ");
    scanf("%c", &ch);

    // Call the function
    result = countChar(str, ch);

    // Print the result
    printf("The character '%c' appears %d times.\n", ch, result);

    return 0;
}
```

Validate and Fix a C Variable Name

Write a C program that does the following:

1. **Reads** a variable name as a string (maximum 50 characters).
2. **Checks** if the name follows basic C naming rules:
   - Must not contain - (dash)
3. **If** the name contains -, **replace** every - with _.
4. After replacement, **Print** the corrected name (after replacements)

Example

- Input: total-sum1
- Output:
  - Corrected: total_sum1
  - Valid: YES

- **Solution**

```c
#include <stdio.h>
#include <string.h>

int containsDash(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == '-') {
            return 1; // found a dash
        }
    }
    return 0; // no dash
}

void replaceDashWithUnderscore(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == '-') {
            s[i] = '_';
        }
    }
}

int main() {
    char name[51];

    printf("Enter a variable name: ");
    fgets(name, sizeof(name), stdin);

    // Remove newline if exists
    name[strcspn(name, "\n")] = '\0';

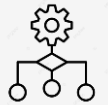
    // If name contains '-', replace with '_'
    if (containsDash(name)) {
        replaceDashWithUnderscore(name);
    }

    printf("Corrected: %s\n", name);



    return 0;
}
```

Find the First Occurrence of a Character



- Write a function that **returns the index of the first occurrence of a character** in a string using pointers.

- **Description:**
  - **Define** a string (`char *str`) and a character (`char ch`).
  - **Write** a function int `findCharIndex(char *str, char ch)` that returns the index of the first occurrence of `ch`.
  - **Print** the index.

- Solution:

```c
// Commented code
#include <stdio.h>

// Function to find the index of the first occurrence of a character in a string using pointers
int findCharIndex(char *str, char ch) {
    int index = 0;  // Initialize the index to 0
    while (*str != '\0') {  // Traverse the string until the null terminator
        if (*str == ch) {  // If the current character matches 'ch'
            return index;  // Return the index of the first occurrence
        }
        str++;  // Move to the next character using pointer arithmetic
        index++;  // Increment the index
    }
    return -1;  // Return -1 if the character is not found
}

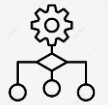
int main() {
    char str[] = "Hello, World!";  // Test string
    char ch = 'o';  // Character to find

    // Find the index of the first occurrence of 'ch' in 'str'
    int index = findCharIndex(str, ch);

    if (index != -1) {
        printf("The character '%c' was found at index: %d\n", ch, index);
    } else {
        printf("The character '%c' was not found in the string.\n", ch);
    }

    return 0;
}
```

Print Each Character Using Pointers



- **Write a function that:**
  - **Accepts a string and prints each character** one by one using pointer arithmetic. Demonstrate the use of pointer manipulation to traverse through the string and print its characters.
  - **Counts and returns the length** of a string manually, using a pointer. You are not allowed to use **strlen()**.

- **Hint:** Use a pointer to iterate through the string until the null terminator **('\0')** is encountered.

- **Solution**

```c
#include <stdio.h>

// Function to print each character of the string using pointer arithmetic
void printString(char *str) {
    while (*str != '\0') {  // Loop until we reach the null terminator
        putchar(*str);  // Print the current character
        str++;  // Move to the next character using pointer arithmetic
    }
    putchar('\n');  // Print a new line after printing the string
}

// Function to count the length of the string manually using pointer
int stringLength(char *str) {
    int length = 0;
    while (*str != '\0') {  // Traverse until null terminator
        length++;  // Increment the length
        str++;  // Move to the next character
    }
    return length;
}

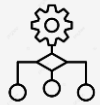
int main() {
    char str[] = "Hello, World!";  // Test string

    // Print each character using pointer arithmetic
    printf("String characters:\n");
    printString(str);

    // Get and print the length of the string manually
    int length = stringLength(str);
    printf("Length of the string: %d\n", length);

    return 0;
}
```

Modify the String



- Write a function that **accepts a string** and **modifies the second character** to 'X' without changing the rest of the string.

Modify the String

- Solution

```c
// Commented code
#include <stdio.h>

// Function that modifies the second character of a string to 'X'
void modifySecondCharacter(char *str) {
    // Check if the string has at least two characters
    if (str != NULL && str[1] != '\0') {
        str[1] = 'X';  // Modify the second character to 'X'
    }
}

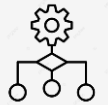
int main() {
    char str[] = "Hello, World!";  // Original string

    printf("Before modification: %s\n", str);  // Print the string before modification

    modifySecondCharacter(str);  // Call the function to modify the second character

    printf("After modification: %s\n", str);  // Print the string after modification
    return 0;
}
```

Count Vowels in a String Using Pointers



- Write a function to **count the number of vowels** in a string using pointers.
- Print the number of vowels.

- **Hint:** Define a string (char *str).Write a function int countVowels(char *str) that counts and returns the number of vowels in the string.

- Solution

```c
//Commented code
#include <stdio.h>

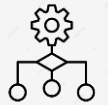
// Function to count the number of vowels in a string using pointers
int countVowels(char *str) {
    int count = 0;
    while (*str != '\0') {  // Traverse until null terminator
        // Check if the character is a vowel (both lowercase and uppercase)
        if (*str == 'a' || *str == 'e' || *str == 'i' || *str == 'o' || *str == 'u' |
            *str == 'A' || *str == 'E' || *str == 'I' || *str == 'O' || *str == 'U')
            count++;  // Increment vowel count
        }
        str++;  // Move to the next character using pointer arithmetic
    }
    return count;  // Return the total count of vowels
}

int main() {
    char str[] = "Hello, World!";  // Sample string

    // Prompt the user to enter a string
//    char str[100];  // Declare a string of sufficient size to hold user input
//    printf("Enter a string: ");
//    fgets(str, sizeof(str), stdin);  // Read a line of input, including spaces

    int vowelCount = countVowels(str);  // Call the function to count vowels
    printf("Number of vowels: %d\n", vowelCount);  // Print the number of vowels
    return 0;
}
```

Find Substring in a String



- Write a function that **finds the first occurrence of a substring** in a string and returns its starting index. If the substring is not found, return -1.

- **Hint:** Use pointer arithmetic to iterate through the string, checking for the starting character of the substring.

Find Substring in a String

- Solution:

```c
#include <stdio.h>
#include <string.h>

// Function to find the first occurrence of a substring in a string
int findSubstringIndex(const char *str, const char *substr) {
    int i, j;
    int strLen = strlen(str);
    int subLen = strlen(substr);

    // Iterate through the string to find the first occurrence
    for (i = 0; i <= strLen - subLen; i++) {
        for (j = 0; j < subLen; j++) {
            if (str[i + j] != substr[j]) {
                break;
            }
        }
        if (j == subLen) {
            return i;  // Return the starting index of the substring
        }
    }
    return -1;  // Return -1 if the substring is not found
}
```

Find Substring in a String

- Solution:
- Main function

```c
int main() {
    char str[100], substr[50];

    // Prompt the user to enter the main string
    printf("Enter the main string: ");
    fgets(str, sizeof(str), stdin);  // Read the string including spaces
    str[strcspn(str, "\n")] = '\0';  // Remove the newline character

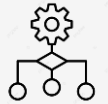
    // Prompt the user to enter the substring
    printf("Enter the substring to search for: ");
    fgets(substr, sizeof(substr), stdin);  // Read the substring
    substr[strcspn(substr, "\n")] = '\0';  // Remove the newline character

    // Call the function to find the first occurrence of the substring
    int index = findSubstringIndex(str, substr);

    // Display the result
    if (index != -1) {
        printf("The substring '%s' was found at index %d.\n", substr, index);
    } else {
        printf("The substring '%s' was not found in the string.\n", substr);
    }

    return 0;
}
```

Remove Spaces from a String

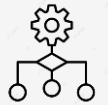


Write a C program that:

1. **Reads** a full line of text (including spaces) into a string (max **100** chars).
2. **Uses** a function `removeSpaces(char s[])` to remove all spaces from the string (shift characters left).
3. **Prints** the updated string.

Self-Study Exercises

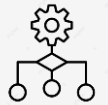
Reverse a String in Place



- Write a function that **reverses a string using pointers** and prints the reversed string.

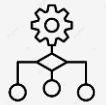
- **Hint:** Use two pointers: one at the beginning of the string and one at the end. Swap the characters pointed to by both pointers, and move the pointers toward each other.

Task : String Comparison



- Write a function that **compares two strings** and returns 0 if they are equal and a non-zero value if they are not. Do not use strcmp().
- **Hint**: Use pointers to compare the characters of both strings one by one..

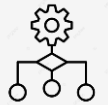
Task 10: Print an Array of Strings



- Write a function that **accepts an array of strings** (an array of char * pointers) and **prints each string on a new line**.

- **Hint:** You can use a for loop and pointer manipulation to access and print each string from the array of strings.

Convert String to Uppercase Using Pointers



- Write a function to **convert all characters of a string to uppercase** using pointers.

- **Description:**
  - Define a string (char *str).Write a function void toUpperCase(char *str) that converts each character of the string to uppercase.
  - Print the modified string.